

Part C — Opinion: Investigating Stale or Irrelevant Answers

Author: Liao Jian Wei

Date: May 2026

Scenario

Six months post-deployment, users report that answers are sometimes outdated or irrelevant even though the source documents are correct and up to date.

Assumption

The system follows the Part A architecture: hybrid BM25 + FAISS retrieval with monthly re-indexing. "Source documents are correct" means the raw files on disk are current; the bug therefore lives in the indexing or retrieval pipeline, not in the documents themselves.

The Three Things I Would Investigate

1. Silent failures in the monthly re-indexing job

Why first: The most common cause of stale answers is a re-indexing job that *appears* to succeed but only partially completed — the process exited cleanly but OOM-killed mid-way, or a file permission error silently skipped a subset of documents.

How specifically:

- Run `md5sum` (or `sha256sum`) on every source document and store the checksums alongside each indexed chunk's metadata at ingest time.
- Write a reconciliation script that re-computes checksums on current source files and diffs them against the stored chunk metadata. Any document whose current checksum does not match its indexed version is a stale-index candidate.
- Cross-check the ingestion job logs for non-zero exit codes, `SIGKILL` events, or per-file error lines that were swallowed by a broad `except Exception: pass`.

If mismatched documents are found, trigger a targeted re-index of only those files rather than a full rebuild.

2. Retrieval recall regression

Why second: Even if all documents are correctly indexed, the retrieval layer may be failing to surface the right chunks — returning topically adjacent but outdated chunks ranked above the correct ones.

How specifically:

- Build a golden evaluation set: 50 queries sampled from real user logs, each manually annotated with the correct source chunk (document ID + approximate paragraph).
- Run retrieval-only (bypass the LLM) against the current indexes and measure **recall@5** — the fraction of queries where the correct chunk appears in the top-5 results.
- If $\text{recall@5} < 0.70$, investigate separately for BM25 (check vocabulary coverage for new terminology added in recent document updates — BM25 is a bag-of-words model and cannot handle vocabulary

drift without re-indexing) and FAISS (verify the embedding model's cosine similarity scores for recently-updated chunks versus older chunks on the same topic).

A recall regression here points to the embedding model no longer representing the evolved corpus well, or BM25 not having been rebuilt despite claimed success.

3. Chunk boundary fragmentation on updated documents

Why third: Documents updated monthly are often not wholesale replacements — editors change a few paragraphs. If the new content is split differently by the chunker (e.g. the policy number moved to a different sentence boundary), the answer-critical fact may now straddle a chunk boundary and appear in neither retrieved chunk in full.

How specifically:

- Sample the 20 queries users flagged as returning outdated answers. For each, retrieve the top-3 chunks and manually verify whether the complete answer exists within a single chunk or is split across adjacent chunks.
- If fragmentation is found in > 30% of sampled failures, increase chunk overlap from 10% to 25% (in the `RecursiveCharacterTextSplitter` `chunk_overlap` parameter) and rebuild the index.
- Optionally switch to a `SentenceWindowNodeParser` (`LlamaIndex`) approach that indexes individual sentences but retrieves a ± 2 -sentence window around each hit, which is more robust to boundary drift without requiring a full overlap increase.

This is investigated third because it requires manual inspection and is harder to fix systematically than the first two.